

Building software that makes a difference

This chapter covers

- The problems that Behavior-Driven Development addresses
- General principles and origins of Behavior-Driven Development
- Activities and outcomes seen in a Behavior-Driven Development project
- The pros and cons of Behavior-Driven Development

This book is about building and delivering software that works well and is easy to change and maintain, but more importantly, it's about building software that provides real value to its users. We want to build software well, but we also need to build software that's worth building.

In 2012, the US Air Force decided to ditch a major software project that had already cost over \$1 billion. The Expeditionary Combat Support System was designed to modernize and streamline supply chain management in order to save billions of dollars and meet new legislative requirements. But after seven years of development, the system had still “not yielded any significant military

capability.”¹ The Air Force estimated that an additional \$1.1 billion would be required to deliver just a quarter of the original scope and that the solution could not be rolled out until 2020, three years after the legislative deadline of 2017.

This happens a lot in the software industry. According to a number of studies, around half of all software projects fail to deliver in some significant way. The 2011 edition of the *Standish Group’s annual CHAOS Report* found that 42% of projects were delivered late, ran over budget, or failed to deliver all of the requested features,² and 21% of projects were cancelled entirely. Scott Ambler’s annual survey on IT project success rates uses a more flexible definition of success, but still found a 30–50% failure rate, depending on the methodologies used.³ This corresponds to billions of dollars in wasted effort, writing software that ultimately won’t be used or that doesn’t solve the business problem it was intended to solve.

What if it didn’t have to be this way? What if we could write software in a way that would let us discover and focus our efforts on what really matters? What if we could objectively learn what features will really benefit the organization and learn the most cost-effective way to implement them? What if we could see beyond what the user asks for and build what the user actually needs?

Organizations are discovering how to do just that. Many teams are successfully collaborating to build and deliver more valuable, more effective, and more reliable software. And they’re learning to do this faster and more efficiently. In this book, you’ll see how. We’ll explore a number of methods and techniques, grouped under the general heading of *Behavior-Driven Development (BDD)*.

BDD is a collaborative development approach where teams use structured conversations about examples and counterexamples of business rules and expected behavior to build a deep, shared understanding of the features that will really benefit the users and the business as a whole. Very often they express these examples in an executable format that acts as the basis for automated acceptance tests that validate the software’s behavior. BDD helps teams focus their efforts on identifying, understanding, and building valuable features that matter to businesses, and it makes sure that these features are well designed and well implemented.

BDD practitioners use conversations around concrete examples of system behavior to help understand how features will provide value to the business. It encourages business analysts, software developers, and testers to collaborate more closely by enabling them to express requirements in a more testable way, in a form that both the development team and business stakeholders can easily understand. BDD tools can help turn

¹ Chris Kanaracus, “Air Force scraps massive ERP project after racking Up \$1 billion in costs,” CIO, November 14, 2012, <https://www.computerworld.com/article/2493041/air-force-scraps-massive-erp-project-after-racking-up-1b-in-costs.html>.

² Whether these figures reflect more on our ability to build and deliver software or on our ability to plan and estimate is a subject of some debate in the Agile development community—see Jim Highsmith’s book *Agile Project Management: Creating Innovative Products*, second edition (Addison-Wesley Professional, 2009).

³ Scott Ambler, “Surveys Exploring the Current State of Information Technology Practices,” <http://www.ambysoft.com/surveys/>.

these requirements into **automated tests** that help guide the developer, verify the feature, and document what the application does.

BDD isn't a software development methodology in its own right. It's not a replacement for Scrum, XP, Kanban, or whatever methodology you're currently using. As you'll see, BDD incorporates, builds on, and enhances ideas from many of these methodologies. And no matter what methodology you're using, there are ways that BDD can help make your life easier.

1.1 BDD from 50,000 feet

So what does BDD bring to the table? Here's a (slightly oversimplified) perspective. Let's say Chris's company needs a new module for its accounting software. When Chris wants to add a new feature, the process goes something like this (see figure 1.1):

- 1 Chris tells a business analyst how he would like the feature to work.
- 2 The business analyst translates Chris's requests into a set of requirements for the developers, describing what the software should do. These requirements are written in English and stored in a Microsoft Word document.
- 3 The developer translates the requirements into code and unit tests—written in Java, C#, or some other programming language—in order to implement the new feature.

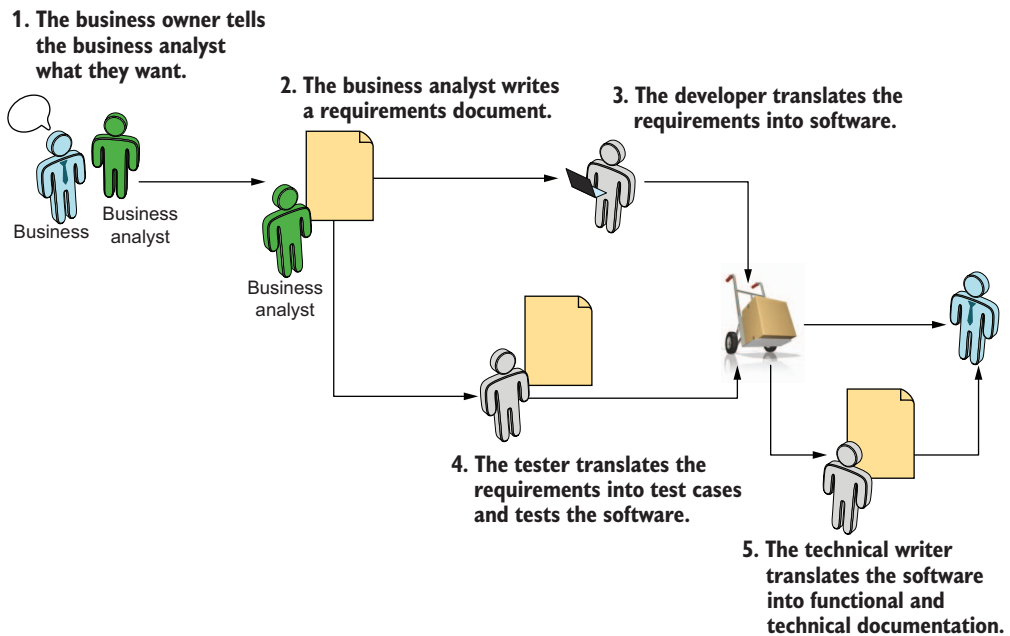


Figure 1.1 The traditional development process provides many opportunities for misunderstandings and miscommunication.

- 4 The tester translates the requirements in the Word document into test cases and uses them to verify that the new feature meets the requirements.
- 5 Documentation engineers then translate the working software and code back into plain English technical and functional documentation.

Along the way there are many opportunities for information to get lost in translation, be misunderstood, or just be ignored. Chances are that the new module itself may not do exactly what was required and that the documentation won't reflect the initial requirements that Chris gave the analyst.

Chris's friend Sarah runs another company that's just introduced BDD. In a team practicing BDD, the business analysts, developers, and testers collaborate to understand and define the requirements (see figure 1.2). They express the requirements in a common language that helps unite and focus the team's efforts. They can even turn these requirements into automated acceptance tests that both specify how the software should behave and also demonstrate that the delivered software behaves as it should. We can see this flow in figure 1.2.

- 1 Like Chris, Sarah talks to Belinda, the business analyst, to get a high-level vision of what she wants. But she doesn't do so alone: she is joined by a developer and a tester who get to hear firsthand what the users really need. To reduce the risk

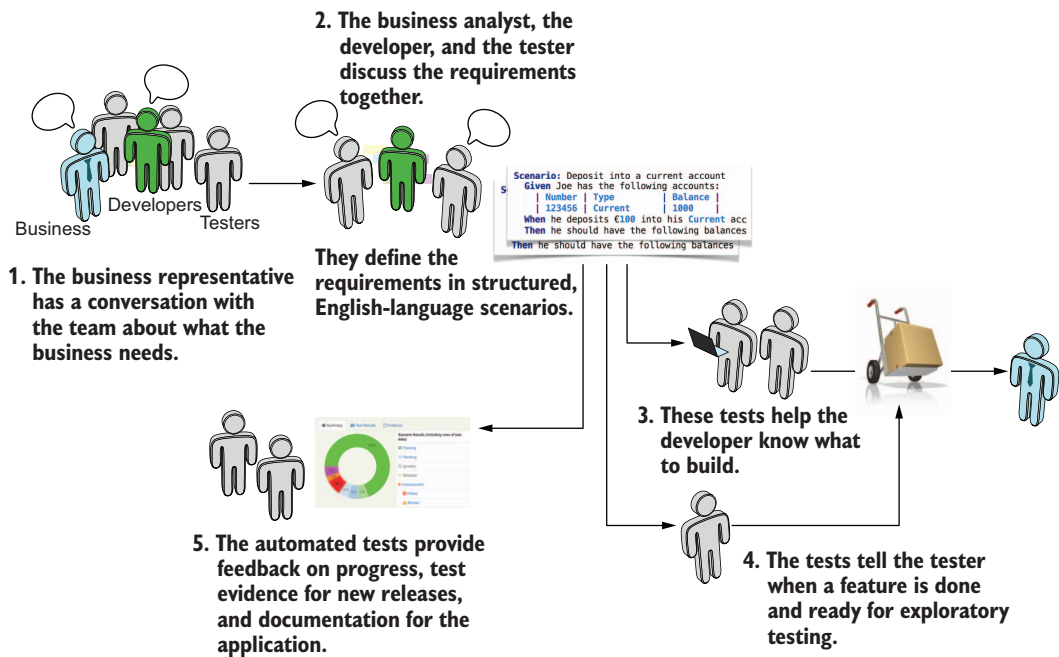


Figure 1.2 BDD uses conversations around business rules and examples, expressed in a form that can be easily automated, to reduce lost information and misunderstandings.

of misunderstandings and hidden assumptions, they talk through examples of what the feature should do and what it shouldn't. They try to articulate the business problem they are trying to solve, the business goal they are aiming for, and what features and capabilities might help achieve this goal.

- 2 Before work starts on the feature, Belinda gets together with the developer and tester who will be working on it, and they have a conversation about the feature. In these conversations, they discuss the key business goals and outcomes of the feature and work through concrete examples and counterexamples to get a deeper understanding of the requirement. Oftentimes, for more important features, Sarah will participate in this conversation as well.

After this conversation, team members write up the key examples and counterexamples in a structured, business-readable format that is quite close to plain English. These examples act both as specifications of the features and as the basis for automated acceptance tests.

- 3 The developers and testers turn these “executable specifications” into automated acceptance tests; these automated tests help guide the development process and determine when a feature is finished.
- 4 When the automated acceptance tests pass, the team has concrete proof that the feature does what was agreed on in phase 2. The tester might use the results of these tests as the starting point for any manual and exploratory testing that needs to be done.
- 5 The automated tests also act as product documentation, providing precise and up-to-date examples of how the system works. Sarah can review the test reports to see what features have been delivered and whether they perform the way she expected.

Compared to Chris's scenario, Sarah's team makes heavy use of conversations and examples to reduce the amount of information lost in translation. Every stage beyond step 2 starts with the specifications written in a structured but business-readable style, which are based on concrete examples provided by Sarah. In this way, a great deal of the ambiguity in translating the client's initial requirements into code, reports, and documentation is removed.

We'll discuss all of these points in detail throughout the rest of the book. You'll learn ways to help ensure that your code is of high quality, solid, well tested, and well documented. You'll learn how to write more effective unit tests and more meaningful automated acceptance criteria. You'll also learn how to ensure that the features you deliver solve the right problems and provide real benefit to the users and the business.

1.2 What problems are you trying to solve?

Software projects fail for many reasons, but the most significant causes fall into two broad categories:

- Not building the software right
- Not building the right software

Figure 1.3 illustrates this in the form of a graph. The vertical axis represents *what* you're building, and the horizontal axis represents *how* you build it. If you perform poorly on the *how* axis, not writing well-crafted and well-designed software, you'll end up with a buggy, unreliable product that's hard to change and maintain. If you don't do well on the *what* axis, failing to understand what features the business really needs, you'll end up with a product that nobody needs.

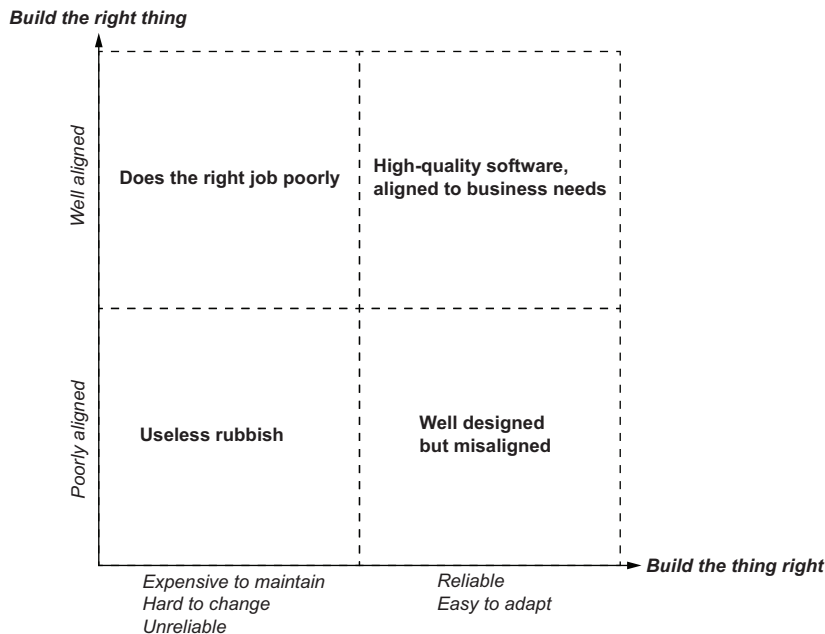


Figure 1.3 Successful projects must both build features well and build the right features.

1.2.1 Building the software right

Many projects suffer or fail because of software quality problems. Although internal software quality is mostly invisible to nontechnical stakeholders, the consequences of poor-quality software can be painfully visible. In our experience, applications that are poorly designed, badly written, or lack well-written, automated tests tend to be buggy, hard to maintain, hard to change, and hard to scale.

We've seen too many applications where simple change requests and new features take too long to deliver. Developers spend more and more time fixing bugs rather than working on new features, which makes it harder to deliver new features quickly. It takes longer for new developers to get up to speed and become productive, simply because the code is hard to understand. It also becomes harder and harder to add new features without breaking existing code. The existing technical documentation (if there is any) is inevitably out of date, and teams find themselves incapable of delivering new features quickly because each release requires a lengthy period of manual testing and bug fixes.

Organizations that embrace high-quality technical practices have a different story to tell. We've seen many teams that adopt practices such as **Test-Driven Development**, **Clean Coding**, **Living Documentation**, and **Continuous Integration** regularly reporting low to near-zero defect rates, as well as code that's much easier to adapt and extend as new requirements emerge and new features are requested. These teams can also add features at a more consistent pace, because the automated tests ensure that existing features won't be broken unknowingly. They implement the features faster and more precisely than other teams because they don't have to struggle with long bug-fixing sessions and unpredictable side effects when they make changes. And the resulting application is easier and cheaper to maintain.

Note that there is no magic formula for building high-quality, easily maintainable software. Software development is a complex field, human factors abound, and techniques such as Test-Driven Development, Clean Coding, and Automated Testing don't automatically guarantee good results. But **studies do suggest a strong correlation between lean and Agile practices and project success rates⁴ when compared to more traditional approaches**. Other studies have found a correlation between Test-Driven Development practices, reduced bug counts,⁵ and improved code quality.⁶ Although it's certainly possible to write high-quality code without practicing techniques such as Test-Driven Development and Clean Coding, teams that value good development practices do seem to succeed in delivering high-quality code more often.

But building high-quality software isn't in itself enough to guarantee a successful project. The software must also benefit its users and business stakeholders.

1.2.2 Building the right software

Software is never developed in a vacuum. Software projects are part of a broader business strategy, and they need to be aligned with business goals if they're to be beneficial

⁴ See, for example, Scott Wambler, "2018 IT Project Success Rates Survey Results," <http://www.ambysoft.com/surveys/success2018.html>.

⁵ See, for example, Nachiappan Nagappan, E. Michael Maximilien, Thirumalesh Bhat, and Laurie Williams, "Realizing quality improvement through test driven development: results and experiences of four industrial teams," https://www.microsoft.com/en-us/research/wp-content/uploads/2009/10/Realizing-Quality-Improvement-Through-Test-Driven-Development-Results-and-Experiences-of-Four-Industrial-Teams-nagappan_tdd.pdf.

⁶ Rod Hilton, "Quantitatively Evaluating Test-Driven Development by Applying Object-Oriented Quality Metrics to Open Source Projects" (PhD thesis, Regis University, 2009), http://www.rodhilton.com/files/tdd_thesis.pdf.

to the organization. At the end of the day, the software solution you deliver needs to help users achieve their goals more effectively. Any effort that doesn't contribute to this end is wasted.

In practice, there's often a lot of waste. In many projects, time and money are spent building features that are never used or that provide only marginal value to the business. According to the Standish Group's CHAOS studies,⁷ on average some 45% of the features delivered into production are never used. Even apparently predictable projects, such as migrating software from a mainframe system onto a more modern platform, have their share of features that need updating or that are no longer necessary. When you don't fully understand the goals that your client is trying to achieve, it's very easy to deliver perfectly functional, well-written features that are of little use to the end user.

On the other hand, many software projects end up delivering little or no real business value. Not only do they deliver features that are of little use to the business, but they fail to even deliver the minimum capabilities that would make the projects viable.

The consequences of not building it right and not building the right thing

The affect of poorly understood requirements and poor code realization isn't just a theoretical concept or a "nice to have;" on the contrary, it's often painfully concrete. In December 2007, the Queensland Health Department kicked off work on a new payroll system for its 85,000 employees. The initial budget for the project was around \$6 million, with a delivery date of August 2008.

When the solution was rolled out in 2010, some 18 months late, it was a disaster. Tens of thousands of public servants were underpaid, overpaid, or not paid at all. Since the go-live date, over 1,000 payroll staff have been required to carry out some 200,000 manual processes each fortnight to ensure that staff salaries are paid.

In 2012, an independent review found that the project had cost the state over \$416 million since going into production and would cost an additional \$837 million to fix. This colossal sum included \$220 million just to fix the immediate software problems that were preventing the system from delivering its core capability of paying Queensland Health staff what they were owed each month.

Building the right software is made even trickier by one commonly overlooked fact: early on in a project, you usually don't know what the right features are.⁸

As we will see in the rest of this book, BDD is a very effective way to address both of these problems. And one of the main ways it does so is by tackling one of the principle causes of risk and overrun in software projects: the team not having enough clarity on what they are supposed to be building.

⁷ The Standish Group's *CHAOS Report 2002* reported a value of 45%, and I've seen more recent internal studies where the figure is around 50%.

⁸ See KPMG, "Review of the Queensland Health Payroll System," 2012, http://delimiter.com.au/wp-content/uploads/2012/06/KPMG_audit.pdf.

1.2.3 The knowledge constraint: Dealing with uncertainty

One fact of life in software development is that there will be things you don't know. Changing requirements are a normal part of every software project. Knowledge and understanding about the problem at hand and about how best to solve it increases progressively throughout the project.

In software development, each project is different. There are always new business requirements to cater to, new technological problems to solve, and new opportunities to seize. As a project progresses, market conditions, business strategies, technological constraints, or simply your understanding of the requirements will evolve, and you'll need to change your tack and adjust your course. Each project is a journey of discovery, where the real constraint isn't time, the budget, or even programmer hours, but your lack of knowledge about what you need to build and how you should build it. When reality doesn't go according to plan, you need to adapt to reality, rather than trying to force reality to fit into your plan. "When the terrain disagrees with the map, trust the terrain" (Swiss Army proverb).

Users and stakeholders will usually know what high-level goals they want to achieve and can be coaxed into revealing these goals if you take the time to ask. They'll be able to tell you that they need an online ticketing system or a payroll solution that caters to 85,000 different employees. And you can get a feel for the scope of the application you might need to build early on in the project.

But the details are another matter entirely. Although users are quick to ask for specific technical solutions to their problems, they're not usually the best placed to know what solution would serve them best, or even what solutions exist. Your team's collective understanding of the best way to deliver these capabilities, as well as the optimal feature set for achieving the underlying business goals, will grow as the project progresses.

As illustrated in figure 1.4, the more prescriptive, plan-based requirements-analysis techniques suppose that you can learn almost all there is to know about a project's requirements, as well as the optimal solution design, very quickly in the early phases of the project. By the end of the analysis phase, the specifications are signed-off on and locked down, and all that remains to do is code.

Of course, reality doesn't always work this way. At the start of the project, a development team will often have only a superficial understanding of the business domain and the goals the users need to achieve. In fact, the job of a software engineering team isn't to know how to build a solution; it's to know how to discover the best way to build the solution.

The team's collective understanding will naturally increase over the duration of the project. You become less ignorant over time. Toward the end of the project, a good team will have built up a deep, intimate knowledge of the user's needs and will be able to proactively propose features and implementations that will be better suited to the particular user base. But this learning path is neither linear nor predictable. It's hard to know what you don't know, so it's hard to predict what you'll learn as the project progresses.

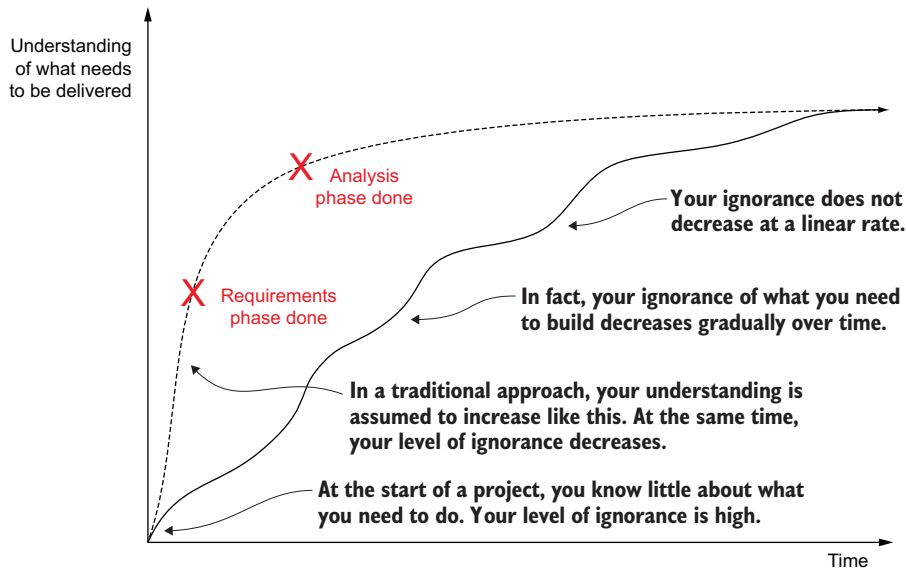


Figure 1.4 At the start of a project, there are many unknowns. You reduce these unknowns as the project progresses, but not in a linear or very predictable way.

For the majority of modern software development projects, the main challenge in managing scope isn't to eliminate uncertainty by defining and locking down requirements as early as possible. The main challenge is to manage this uncertainty in a way that will help you progressively discover and deliver an effective solution that matches up with the underlying business goals behind a project. As you'll see, one important benefit of BDD is that it provides techniques that can help you manage this uncertainty and reduce the risk that comes with it.

1.3 *Is BDD right for your projects?*

BDD works well with other Agile methodologies such as Scrum, but it can also be used with lean approaches such as Kanban. It is not a separate methodology, but more a collection of practices that helps teams discover and understand business needs more quickly and more effectively and get automated feedback on whether these needs have indeed been met by the features they build. For example, a Scrum team using BDD will work in much the same way as an ordinary Scrum team, but they will apply BDD practices during their backlog refinement sessions to get more clarity on the features they need to build. They will also pay more attention to in-sprint automation and try to write automated tests for the acceptance criteria for the features they deliver during the sprint. A Scrum team practicing BDD will also want to add "passing automated acceptance tests" to the definition of their user stories.

BDD works well for any kind of requirements discovery, both in green fields projects and in ones that are already underway. The examples in this book focus mostly on

building new features and new applications (with the exception of chapters 14 and 15, where we look specifically at working with legacy applications). This is by design, to make the domains easier to understand and the examples more engaging. However, BDD is also very effective for existing applications and complex domains. Both authors spend much of their time working in large financial organizations on complex projects. In fact, the techniques you will learn in this book apply to any domain where the requirements are not trivial, where assumptions need to be uncovered, and where complexity and uncertainty lie underneath the surface of each story, which covers almost every project we have ever worked on.

1.4 What you will learn in this book

This book gives you both an understanding of the theoretical foundations of BDD, and hands-on knowledge about what you need to do to introduce BDD into your own organization. You will learn the following:

- How to get clarity on real user requirements using collaborative techniques and Agile requirements to discover techniques such as Example Mapping and Feature Mapping
- How to record these requirements in an executable format (*executable specifications*) that can act as automated tests and be integrated into your build process
- How to automate these executable specifications using Java or JavaScript, and using tools such as Cucumber
- How to use these executable specifications to produce living documentation that both verifies and documents the features you deliver

Summary

- The developers of successful projects need to build software that's reliable and bug free (build the software right) and to build features that deliver real value to the business (build the right software).
- BDD is a collaborative development approach where teams use structured conversations about examples and counterexamples of business rules and expected behavior to build a deep, shared understanding of what features will benefit users. Very often they express these examples in an executable format that acts as the basis for automated acceptance tests that validate the software's behavior.
- BDD practitioners use conversations about concrete examples to build a common understanding of what features will deliver real value to the organization.
- These examples form the basis of the acceptance criteria that developers use to determine when a feature is done.

In the next chapter, we'll look at the origins of BDD and learn about the key steps of BDD process in more detail.